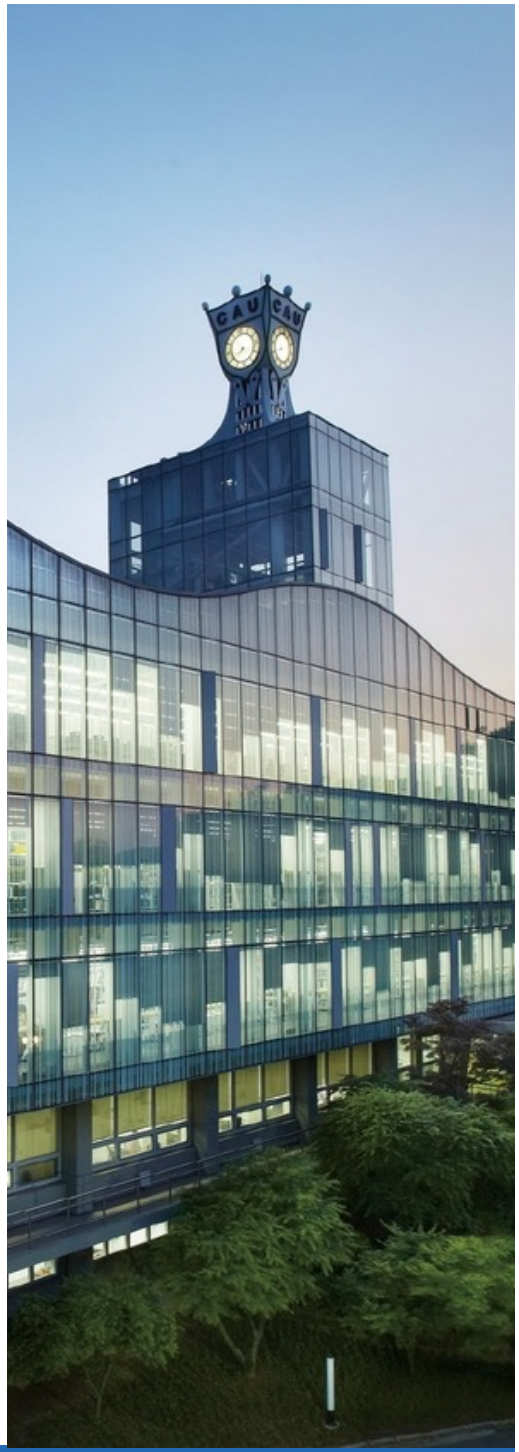
Principles of Programming Languages

Lecture 2
Preliminaries (기본사항)

Jinsung Kim, Ph.D.
High-Performance Computing (HPC) Lab
Computer Science and Engineering
Chung-Ang University



Topics

- Reasons for Studying Concepts of Programming Languages
- Programming Domains
- Language Evaluation Criteria
- Influences on Language Design
- Language Categories
- Language Design Trade-Offs
- Implementation Methods
- Programming Environments

Reasons for Studying Concepts of PL (1/3)

- **Increased ability to express ideas**
  - The language places **limits on** the kinds of control structures, data structures, and abstractions programmers can use;
  - The forms of algorithms they can construct are **limited**
  - **Encourage** users to use another language's features and constructs
- **Improved background for choosing appropriate languages**
  - The language with the features that best address the problem
- **Increased ability to learn new languages**
  - A thorough understanding of the fundamental concepts of languages helps to see how these concepts are incorporated into the design of the language being learned

Reasons for Studying Concepts of PL (2/3)

- **Better understanding of significance of implementation (구현의 중요성)**
  - An understanding of implementation issues leads to an understanding of why languages are designed the way they are
  - Use a language more efficiently (intelligently)
- **Better use of languages that are already known**
  - Can learn about previously unknown and unused parts of the languages
- **Overall advancement of computing**
  - Programming languages have evolved in response to new computing needs, hardware advancements, and shifts in how we approach problem-solving

Reasons for Studying Concepts of PL (3/3)

- **Deeper understanding of language design and compiler construction**
  - **Language design principles:** Concepts such as type systems, scoping, binding, and ***runtime environments*** are key to understanding how programming languages work. This understanding is important when designing new languages or tools
  - **Compiler optimization:** Knowledge of how languages are compiled or interpreted enables engineers to write more efficient code and even contribute to language development or compiler design
- **Increased code maintainability**
  - Maintainable code: Learning concepts such as ***modularity***, ***encapsulation***, and ***abstraction*** improves the ability to write maintainable, reusable, and scalable code

Programming Domains (1/2)

- **Scientific applications**
  - Large numbers of floating-point computations; use of arrays
  - Fortran (**FOR**mula **TRAN**slation)
- **Business applications**
  - Produce reports, use decimal numbers and characters
  - COBOL (**CO**mmon **B**usiness-**O**riented **L**anguage)
- **Artificial intelligence**
  - Symbols rather than numbers manipulated; use of linked lists
  - LISP (**LIS**t **P**rocessing)
    - Symbolic Processing
    - **Homoiconicity** (Code as Data): Building self-modifying code and meta-programming

Programming Domains (2/2)

- **Systems programming**
  - Need efficiency because of continuous use
  - C

- **Web Software**
  - Eclectic collection of languages: markup (e.g.,HTML), scripting (e.g., PHP), general-purpose (e.g., Java)

Language Evaluation Criteria

- **Readability (판독성)**
  - The ease with which programs can be read and understood
- **Writability (작성력)**
  - The ease with which a language can be used to create programs
- **Reliability (신뢰성)**
  - Conformance to specifications (i.e., performs to its specifications under all conditions)
- **Cost (비용)**
  - The ultimate total cost of training programmers, and writing/compiling/executing programs

Language Evaluation Criteria

언어 평가 기준과 언어 평가 기준에 영향을 주는 특성들

Table 1.1 Language evaluation criteria and the characteristics that affect them

	CRITERIA		
Characteristic	READABILITY	WRITABILITY	RELIABILITY
Simplicity 단순성	•	•	•
Orthogonality 직교성	•	•	•
Data types	•	•	•
Syntax design	•	•	•
Support for abstraction		•	•
Expressivity		•	•
Type checking			•
Exception handling			•
Restricted aliasing			•

Characteristics for Readability (1/4)

- **Why readability (판독성) matters?**
  - Maintenance was recognized as a major part of the SW life cycle
  - Ease of maintenance is determined largely by the readability
  - A distinct crossover from a focus on machine orientation to a focus on human orientation
  - Readability must be considered in the context of the problem domain

- **Overall simplicity (전반적인 단순성)**
  - A language with a small number of features and constructs
  - Minimal feature multiplicity (특징 다중성)
    - Have more than one way to accomplish a particular operation
    - e.g., `cnt=cnt+1; cnt+=1; cnt++; ++cnt;`
  - Minimal operator overloading (연산자 중복)
    - Operator overloading: a single operator symbol has more than one meaning

Characteristics for Readability (2/4)

- **Orthogonality (직교성)**
  - Every possible combination of primitive constructs is legal and meaningful to build another control and data structures
  - More orthogonality → Fewer rule exceptions → Higher degree of regularity → Easier to learn/read
  - Example of unorthogonal design in C
    - Records(`struct`) can be returned from functions, but arrays cannot
    - An array element can be any data type except `void` or a function
  - Example of orthogonal design (the VAX): `ADDL oprd_1, oprd_2`
    - In VAX, operands can be registers or memory cells
  - Too much orthogonality can also cause problems
    - `x := if a > b then a else b fi;`
    - `x := (if a > b then a := b else b := a fi);`

Characteristics for Readability (3/4)

- **Data types (데이터 타입)**
  - The presence of adequate facilities (충분한 장치) for defining data types and data structures is good for readability
  - Example: No Boolean type in the language
    - `timeOut = 1` vs `timeOut = false`
    - C89 (ANSI C) and C90 (ISO 90) do not have a native Boolean type
      - Must use integers (0 for false, non-zero for true)
    - C99 introduced the stdbool.h header, which defines a Boolean type
      - bool, true, false

Characteristics for Readability (4/4)

- **Syntax design (구문 설계)**
  - The syntax, or form, of the elements of a language has a significant effect on the readability of programs
  - Examples of syntactic design choices that affect readability
    - Special words (특수어) (e.g., `while`, `for`)
  - The method of forming compound statements
    - C and its descendants use braces "`{`", "`}`" to specify compound statements
    - In ADA, use '`end if`', '`end loop`', etc.
  - Form and meaning (형식과 의미): semantics should follow directly from syntax
    - In C, the meaning of the reserved word (예약어) `static` depends on the context of its appearance
      - If declared inside a function, the variable is created compile time (not created when the function is called.)
      - If declared outside all functions, the variable is visible only in the file

Characteristics for Writability (1/2)

- **A measure of how easily a language can be used to create programs for a chosen program domain**
  - Most of the language characteristics that affect readability also affect writability
  - The context of the target problem domain of a language

- **Simplicity and orthogonality (단순성과 직교성)**
  - Few constructs, a small number of primitives, a small set of rules for combining them
  - Reduce a misuse of some features and a disuse of others
  - Pros and cons
    - Need to learn only a simple set of primitive constructs
    - Due to too much orthogonality, errors in program can go undetected

Characteristics for Writability (2/2)

- **Expressivity (표현력)**
  - Using very powerful operators, a great deal of computation can be accomplished with a very small program (e.g., `A+ . xB` in APL)
  - Provide relatively convenient ways of specifying computations (e.g., in C, the notation `count++` is more convenient and shorter than `count = count + 1`)

Characteristics for Reliability

- **Reliable if it performs to its specifications under all conditions**
- **Language features affecting the reliability**
  - Type checking (타입 검사)
    - The earlier errors in programs are detected, the less expensive it is to make the required repairs
      - Compile-time vs. Run-time
    - Java checks the types of nearly all variables and expressions at compile time
  - Exception handling (예외 처리)
    - Intercept run-time errors and take corrective measures (시정조치)
  - Aliasing (별칭)
    - Presence of two or more distinct referencing methods for the same memory location, resulting in reduced reliability
  - Readability and writability (판독성과 작성력)
    - The easier a program is to read or to write, the more likely it is to be correct

Characteristics for Cost

- **Training programmers to use the language**
- **Writing programs in a language**
- **Executing programs**
  - Compiling programs
  - Optimization (최적화)
- **Reliability: poor reliability leads to high costs if the software fails in a critical system, such as a nuclear power plant**
  - Mars Pathfinder Priority Inversion Failure
    - The low-priority task was holding a resource needed by the high-priority task
    - In July 2024, Azure and Microsoft 365 affected aviation and telecommunications industries
- **Maintaining programs for both corrections and modifications to add new functionality**

Characteristics for Others

- **Portability (이식성)**
  - The ease with which programs can be moved from one implementation to another
  - Most strongly influenced by the degree of standardization of the language
- **Generality (일반성)**
  - The applicability to a wide range of applications
- **Well-definedness (분명성)**
  - The completeness and precision of the language's official definition

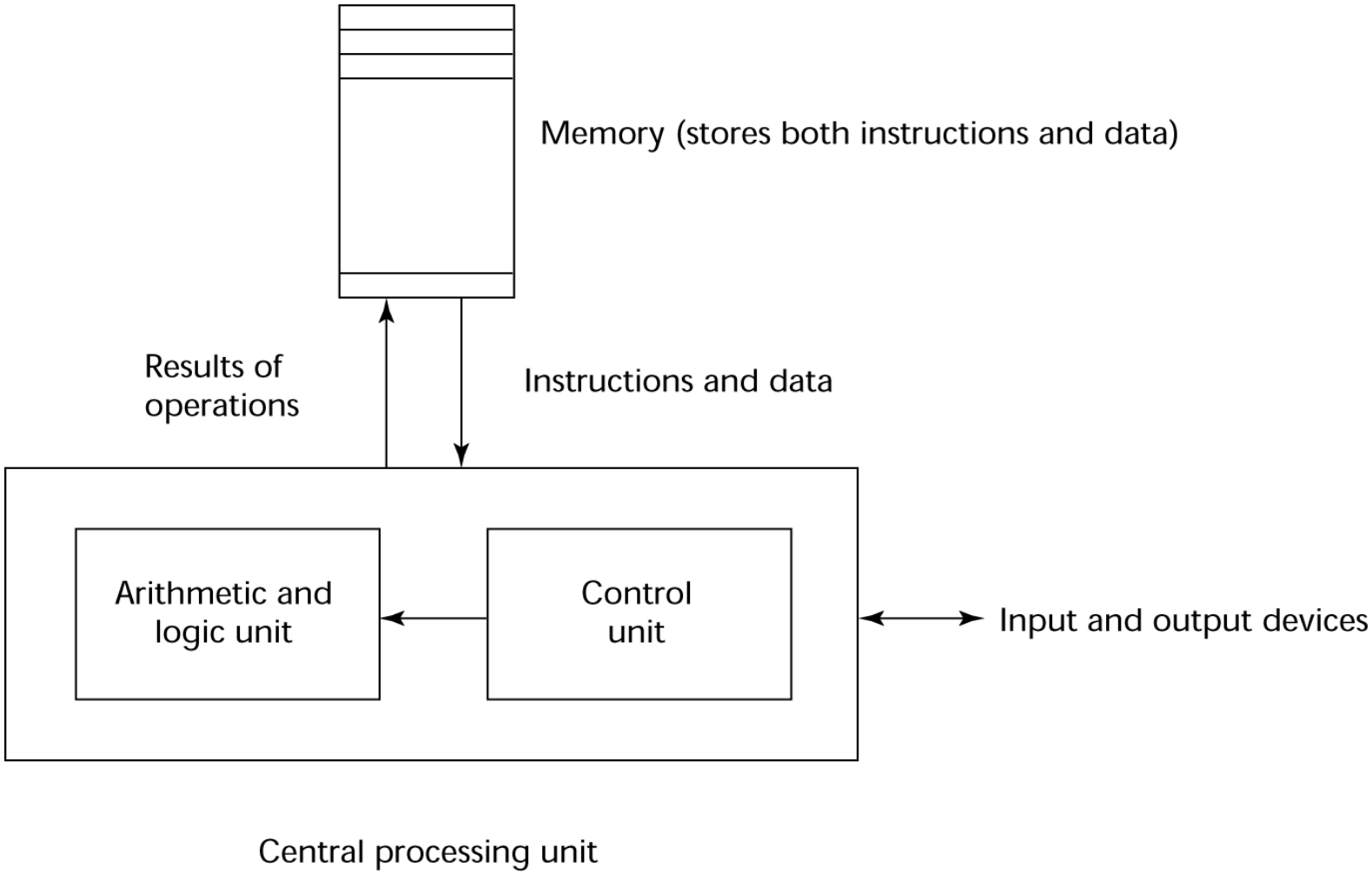
Influences on Language Design

- **Computer architecture (컴퓨터 구조)**
  - Languages are developed around the von Neumann architecture
  - These languages are called imperative languages (명령형 언어)
- **Program design methodologies (프로그램 설계 방법론)**
  - New software development methodologies (e.g., object-oriented software development) led to new programming paradigms and by extension, new programming languages
    - Top-down design (하향식 설계)
    - Procedure-oriented program (절차 지향)
    - Data-oriented program (데이터 지향)
    - Object-oriented program (객체 지향)
      - Inheritance, dynamic method binding
    - …

Computer Architecture Influence

- **Well-known computer architecture: Von Neumann (폰 노이만 구조)**
- **Imperative languages, most dominant, because of von Neumann computers**
  - Data and programs stored in memory
  - Memory is separate from CPU
  - Instructions and data are piped from memory to CPU
  - Basis for imperative languages
    - Variables model memory cells
    - Assignment statements model piping
    - Iteration is efficient

The von Neumann Architecture (1/2)



Von Neumann Bottleneck

- **The transfer speed of a bus between a computer's memory and its processor usually determines the speed of a computer**
- **Program instructions often can be executed much faster than the speed of the transfer; the transfer speed thus results in a bottleneck**
- **The bus speed becomes a bottleneck known as the *von Neumann* bottleneck**
  - It is the primary limiting factor in the speed of computers

The von Neumann Architecture (2/2)

- **Fetch-execute cycle (on a von Neumann architecture computer)**

```
initialize the program counter
repeat forever
     fetch the instruction pointed by the counter
     increment the counter
     decode the instruction
     execute the instruction
end repeat
```

Programming Methodologies Influences

- **1950s and early 1960s: Simple applications; worry about machine efficiency**
- **Late 1960s: People efficiency became important; readability, better control structures**
  - Structured programming
  - Top-down design and step-wise refinement
- **Late 1970s: from Procedure-oriented to data-oriented**
  - Data abstraction
- **Middle 1980s: Object-oriented programming**
  - Data abstraction + inheritance + polymorphism

Language Categories (1/2)

- **Imperative (명령형)**
  - Central features are variables, assignment statements, and iteration
  - Include languages that support object-oriented programming
  - Include scripting languages
  - Include the visual languages
  - Examples: C, Java, Perl, JavaScript, Visual BASIC .NET, C++
- **Functional (함수형)**
  - Main means of making computations is by applying functions to given parameters
    - Immutability (불변성)
    - Pure Function (순수 함수)
  - Examples: LISP, Scheme, ML, F#

Language Categories (2/2)

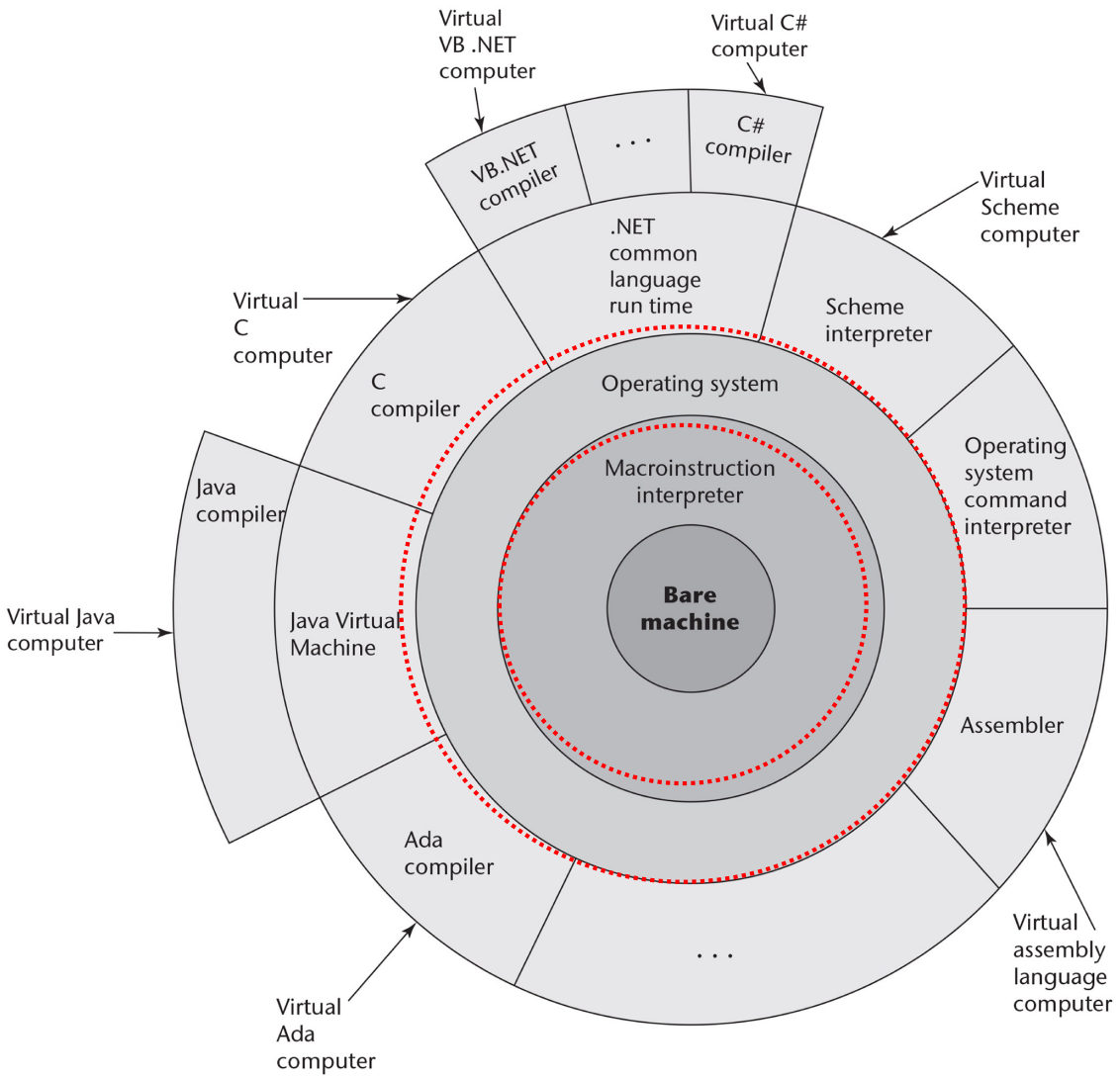
- **Logic (논리형)**
  - Rule-based (rules are specified in no particular order)
  - Example: Prolog
- **Markup/programming hybrid**
  - Markup languages extended to support some programming
  - Examples: JSTL(Java Server pages standard Tag Library), XSLT(eXtensible Stylesheet Language Transformations)

Language Design Trade-Offs

- **"There are so many important but conflicting criteria, that their reconciliation (절충) and satisfaction (만족) is a major engineering task." (Hoare (1973))**
  - Reliability vs. cost of execution
    - Example: Java demands all references to array elements be checked for proper indexing, which leads to increased execution costs
  - Readability vs. writability
    - Example: APL provides many powerful operators (and a large number of new symbols), allowing complex computations to be written in a compact program but at the cost of poor readability
  - Writability (flexibility) vs. reliability
    - Example: C++ pointers are powerful and very flexible but are unreliable
- **Design of a programming language requires many compromises and trade-offs**

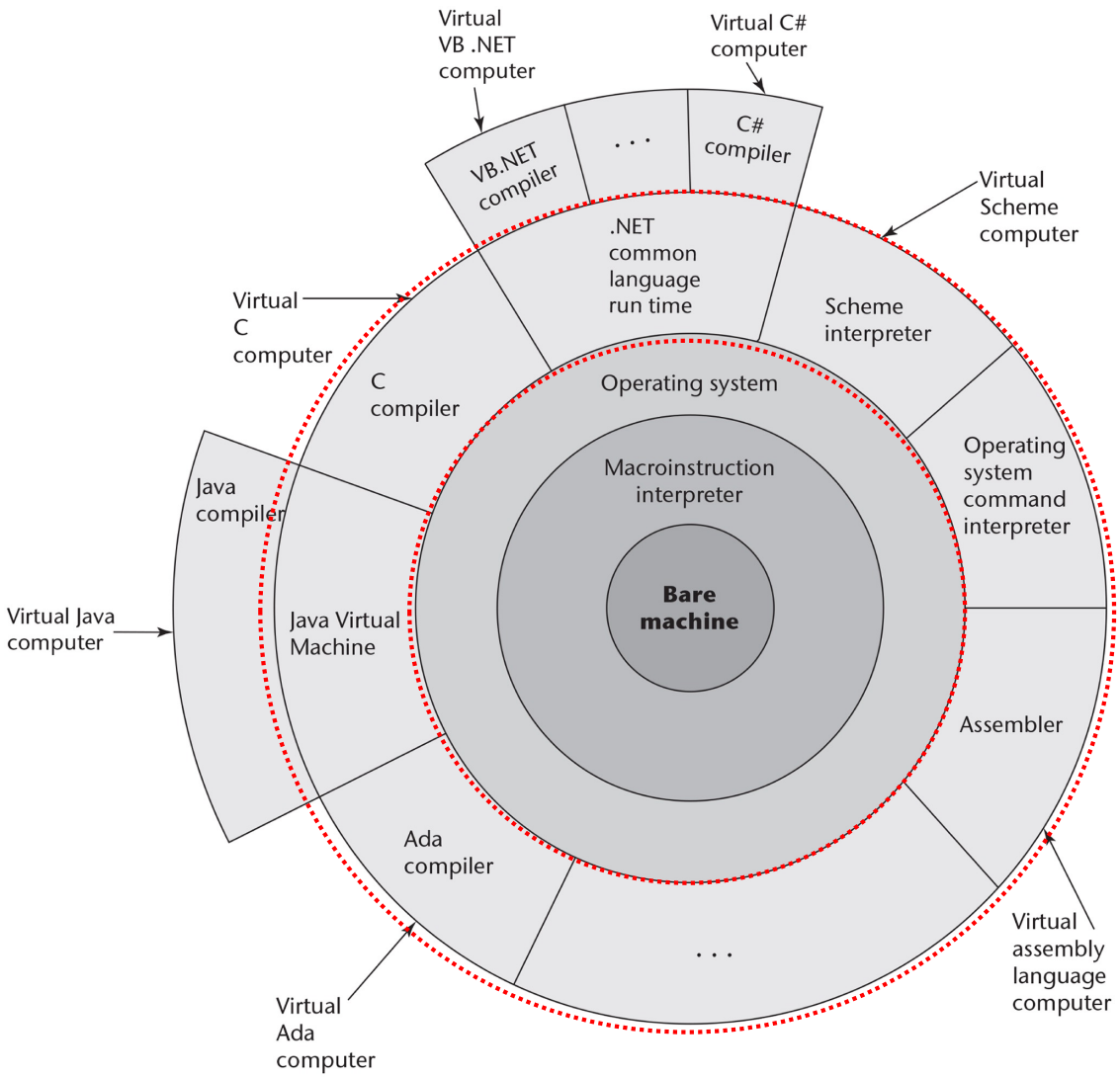
Layered View of Computer

- **The operating system and language implementation are layered over machine interface of a computer**



Layered View of Computer

- **The operating system and language implementation are layered over machine interface of a computer**



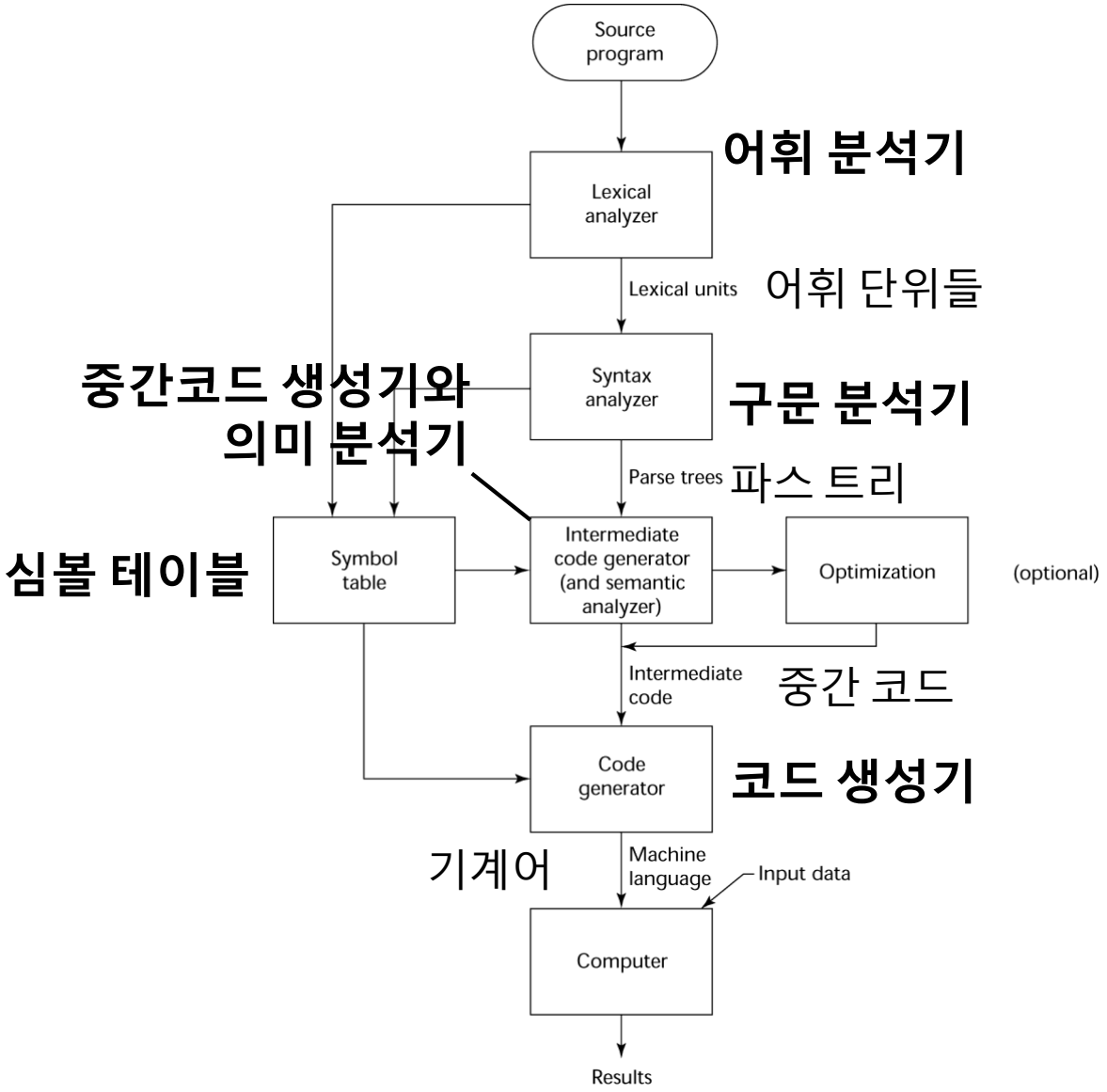
Implementation Methods

- **Compilation (컴파일)**
  - Programs are translated into machine language; includes JIT systems
  - Use: Large commercial applications
- **Pure Interpretation (순수 해석)**
  - Programs are interpreted by another program known as an interpreter
  - Use: Small programs or when efficiency is not an issue
- **Hybrid Implementation Systems (혼합형 구현 시스템)**
  - A compromise between compilers and pure interpreters
  - Use: Small and medium systems when efficiency is not the first concern

Compilation

- **Translate high-level program (source language, 원시 언어) into machine code (machine language, 기계어)**
- **Slow translation, fast execution**
- **Compilation process has several phases:**
  - **Lexical analysis:** converts characters in the source program into lexical units
  - **Syntax analysis:** transforms lexical units into parse trees which represent the syntactic structure of program
  - **Semantics analysis:** generate intermediate code
  - **Code generation:** machine code is generated

The Compilation Process



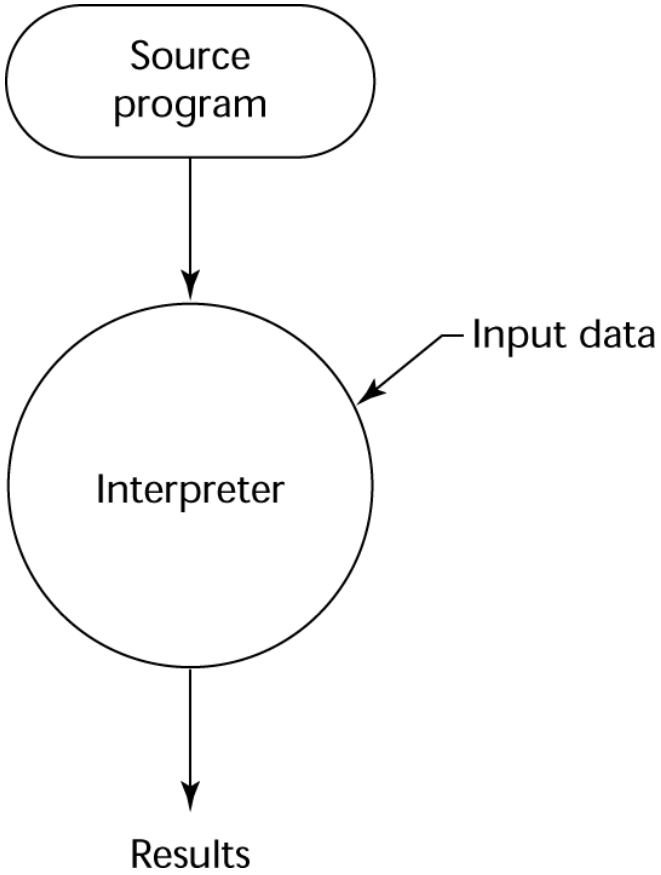
Additional Compilation Terminologies

- **Load module (executable image) // 적재 모듈 (실행가능 이미지)**
  - The output of linking and loading that will be loaded to main memory for execution by a loader
- **Linking and loading (or just linking) // 링킹과 적재**
  - The process of collecting system programs (libraries) and linking them to a user program (object code)

Pure Interpretation

- **No translation**
- **Easier implementation of programs** (run-time errors can easily and immediately be displayed)
- **Slower execution** (10 to 100 times slower than compiled programs)
- **Often requires more space**
- **Now rare for traditional high-level languages**
- **Significant comeback with some Web scripting languages** (e.g., JavaScript, PHP)

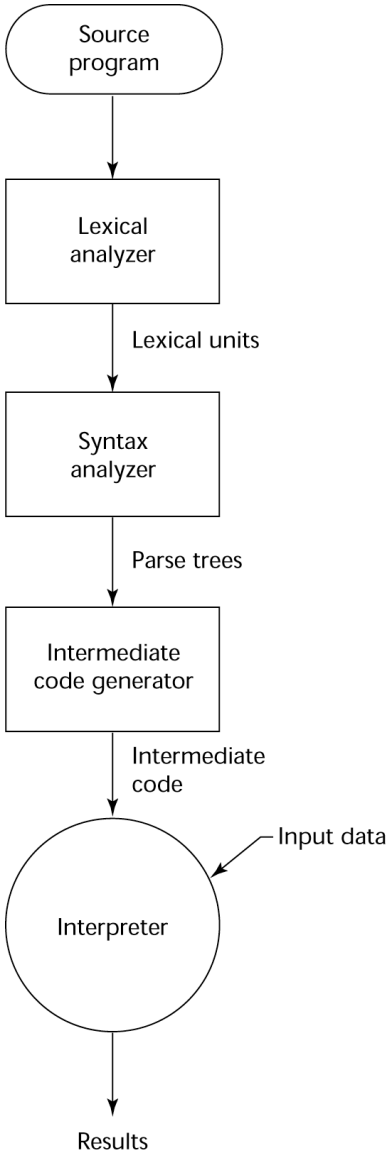
Pure Interpretation Process



Hybrid Implementation Systems

- **A compromise between compilers and pure interpreters**
- **A high-level language program is translated to an intermediate language that allows easy interpretation**
- **Faster than pure interpretation**
- **Examples**
  - Perl programs are partially compiled to detect errors before interpretation
  - Initial implementations of Java were hybrid; the intermediate form, byte code, provides portability to any machine that has a byte code interpreter and a run-time system (together, these are called Java Virtual Machine)

Hybrid Implementation Process



Just-in-Time (JIT) Implementation Systems

- Initially translate programs to an intermediate language
- Then compile the intermediate language of the subprograms into machine code when they are called during execution
- The machine code version is kept for subsequent calls
- JIT systems are widely used for Java programs
- .NET languages are implemented with a JIT system
- In essence, JIT systems are delayed compilers

Preprocessors (전처리기)

- **Preprocessor macros (instructions) are commonly used to specify that code from another file is to be included**
- **A preprocessor processes a program immediately before the program is compiled to expand embedded preprocessor macros**
- **A well-known example: C preprocessor**
  - `#include "myLib.h"` copies the contents of `"myLib.h"` into the program at the position of the #include
  - `#define max(A, B) ((A) > (B) ? (A) : (B))`

Programming Environments

- **A collection of tools used in software development**
  - Language Processors, Editors, Debugger, Build-Tools, Libraries, IDE, OS, etc.
- **UNIX**
  - An older operating system and tool collection
  - Nowadays often used through a GUI (e.g., CDE, KDE, or GNOME) that runs on top of UNIX
- **Microsoft Visual Studio.NET**
  - A large, complex visual environment
  - Used to build Web applications and non-Web applications in any .NET language
- **NetBeans**
  - Related to Visual Studio.NET, except for applications in Java

Summary

- **The study of programming languages is valuable for a number of reasons:**
  - Increase our capacity (능력/역량) to use different constructs
  - Enable us to choose languages more intelligently
  - Makes learning new languages easier
- **Most important criteria for evaluating programming languages include:**
  - Readability, writability, reliability, cost
- **Major influences on language design have been machine architecture and software development methodologies**
- **The major methods of implementing programming languages**
  - Compilation, pure interpretation, and hybrid implementation

Q & A